ArtExtract_2

March 13, 2026

1 ArtExtract (Model for finding similarity in paintings)

Mohit Deharkar | Undergrad @ IIT Jodhpur

This notebook implements the pieces of work as demanded in the Task 2

Using the ***objects.csv*** and ***published_images.csv*** data files from the National Art Gallery Dataset

[1]:
```
import pandas as pd
```

[61]:
```
objects = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/ArtExtract/
↪objects.csv')
published_images = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/
↪ArtExtract/published_images.csv')
```

```
/tmp/ipykernel_315/2614923436.py:1: DtypeWarning: Columns (24) have mixed types.
Specify dtype option on import or set low_memory=False.
  objects = pd.read_csv('/content/drive/MyDrive/Colab
Notebooks/ArtExtract/objects.csv')
```

This is how the raw data looks like

[62]:
```
objects[:5]
```

[62]:
```
   objectid                                  uuid  accessioned accessionnum  \
0         0  b5dc500c-be86-4b05-93fa-59a25d8775fb            1   1937.1.2.c
1         1  59e0e07c-208e-4d24-a3d3-f4a56d20d6d3            1     1937.1.3
2         2  95cfbfd3-5947-4834-b1cf-bf92ee1a7340            1   1937.1.4.a
3         3  3277a6d0-d92e-4684-9d86-2be2cbe3c5f2            1   1937.1.4.b
4         4  63225dec-d421-4ece-9f2a-335dd2dfeea1            1   1937.1.4.c

   locationid                                                title  \
0         NaN                                     Saint James Major
1         NaN                  Saint Paul and a Group of Worshippers
2      8198.0  Saint Andrew and Saint Benedict with the Archa…
3      8198.0  Madonna and Child Enthroned with Twelve Angels…
4      8198.0  Saint Bernard and Saint Catherine of Alexandri…

            displaydate  beginyear  endyear visualbrowsertimespan  …  \
```

```
0              c. 1310     1310.0   1310.0          1300 to 1400  …
1                 1333     1333.0   1333.0          1300 to 1400  …
2  shortly before 1387     1387.0   1387.0          1300 to 1400  …
3  shortly before 1387     1387.0   1387.0          1300 to 1400  …
4  shortly before 1387     1387.0   1387.0          1300 to 1400  …

   parentid isvirtual departmentabbr portfolio series volume watermarks  \
0      34.0         0          CIS-R       NaN    NaN    NaN        NaN
1       NaN         0          CIS-R       NaN    NaN    NaN        NaN
2  206122.0         0          CIS-R       NaN    NaN    NaN        NaN
3  206122.0         0          CIS-R       NaN    NaN    NaN        NaN
4  206122.0         0          CIS-R       NaN    NaN    NaN        NaN

     lastdetectedmodification wikidataid customprinturl
0   2023-05-09 17:01:03.48-04  Q20172973            NaN
1  2024-02-12 16:33:23.167-05  Q20173083            NaN
2   2024-03-08 14:32:09.57-05  Q20173290            NaN
3   2024-03-08 14:32:09.57-05  Q20173280            NaN
4   2024-03-08 14:32:09.57-05  Q20173299            NaN

[5 rows x 31 columns]
```

[63]:
```
published_images
```

[63]:
```
                                        uuid  \
0       00007f61-4922-417b-8f27-893ea328206c
1       0000bd8c-39de-4453-b55d-5e28a9beed38
2       0001668a-dd1c-48e8-9267-b6d1697d43c8
3       0002e210-8bc2-4471-bb0b-e7bb51d01a40
4       00032658-8a7a-44e3-8bb8-df8c172f521d
…                                          …
130933  fffdeb15-ffd1-4fa7-892f-6883be3b9f3c
130934  fffe6d61-e686-478d-bafb-ddae9a48c4de
130935  ffff4903-aaf2-4a8f-9f09-60bddc65ddd7
130936  ffffc381-9fa3-470d-bfc2-441c643c6039
130937  ffffdd66-757e-499b-aa2c-c97e1d437da6

                                                iiifurl  \
0       https://api.nga.gov/iiif/00007f61-4922-417b-8f…
1       https://api.nga.gov/iiif/0000bd8c-39de-4453-b5…
2       https://api.nga.gov/iiif/0001668a-dd1c-48e8-92…
3       https://api.nga.gov/iiif/0002e210-8bc2-4471-bb…
4       https://api.nga.gov/iiif/00032658-8a7a-44e3-8b…
…                                                     …
130933  https://api.nga.gov/iiif/fffdeb15-ffd1-4fa7-89…
130934  https://api.nga.gov/iiif/fffe6d61-e686-478d-ba…
130935  https://api.nga.gov/iiif/ffff4903-aaf2-4a8f-9f…
```

```
130936  https://api.nga.gov/iiif/ffffc381-9fa3-470d-bf…
130937  https://api.nga.gov/iiif/ffffdd66-757e-499b-aa…

                                            iiifthumburl viewtype  sequence  \
0       https://api.nga.gov/iiif/00007f61-4922-417b-8f…  primary         0
1       https://api.nga.gov/iiif/0000bd8c-39de-4453-b5…  primary         0
2       https://api.nga.gov/iiif/0001668a-dd1c-48e8-92…  primary         0
3       https://api.nga.gov/iiif/0002e210-8bc2-4471-bb…  primary         0
4       https://api.nga.gov/iiif/00032658-8a7a-44e3-8b…  primary         0
…                                                     …      …       …
130933  https://api.nga.gov/iiif/fffdeb15-ffd1-4fa7-89…  primary         0
130934  https://api.nga.gov/iiif/fffe6d61-e686-478d-ba…  primary         0
130935  https://api.nga.gov/iiif/ffff4903-aaf2-4a8f-9f…  primary         0
130936  https://api.nga.gov/iiif/ffffc381-9fa3-470d-bf…  primary         0
130937  https://api.nga.gov/iiif/ffffdd66-757e-499b-aa…  primary         0

        width  height  maxpixels  openaccess                 created  \
0        3365    4332        NaN           1  2013-07-05 15:41:08-04
1        3500    4688        NaN           1  2013-08-05 14:31:59-04
2        3446    4448        NaN           1  2014-01-02 14:50:50-05
3        2000    1571      900.0           0  2009-12-10 17:28:05-05
4        2674    3798        NaN           1  2010-10-13 15:37:25-04
…        …     …         …          …                     …
130933   5321    5714      900.0           0  2025-06-05 15:30:56-04
130934   4000    1635        NaN           1  2016-05-10 16:23:54-04
130935   3403    3405      900.0           0  2011-12-22 15:12:59-05
130936   2861    4000      900.0           0  2016-02-03 20:01:19-05
130937   8596   11459        NaN           1  2020-02-11 11:55:29-05

                          modified  depictstmsobjectid assistivetext
0           2025-09-22 14:10:53-04               17387           NaN
1           2025-09-22 14:10:57-04               19245           NaN
2           2025-09-22 14:11:18-04               23830           NaN
3       2026-01-01 07:14:56.937-05              232817           NaN
4           2026-02-11 09:02:19-05                 713           NaN
…                               …                 …            …
130933      2025-09-22 14:10:43-04              221706           NaN
130934      2026-01-22 16:32:44-05              169631           NaN
130935      2026-02-11 18:13:17-05               54273           NaN
130936      2026-02-17 17:08:51-05              165884           NaN
130937      2026-01-21 15:42:11-05              221645           NaN

[130938 rows x 13 columns]
```

[5]:
```
published_images = published_images[['uuid', 'iiifurl', 'depictstmsobjectid',␣
↪'width', 'height']]
objects = objects[['objectid', 'title', 'attribution', 'classification']]
```

Now, i've merged the two datasets on the bases of same object ids in both the datasets

[ ]:
```
## merging the dataset on basis of object id
merged_df = pd.merge(objects, published_images, left_on='objectid',␣
↪right_on='depictstmsobjectid', how='inner')
## we will use only paintings (ignoring photograph, portfolio etc..)
merged_df = merged_df[merged_df['classification'] == 'Painting']
```

[7]:
```
## creating the subset of the dataset
subset_df = merged_df.sample(500)
```

[8]:
```
image_urls = subset_df['iiifurl'].tolist()
```

[ ]:
```
import os
import requests
from concurrent.futures import ThreadPoolExecutor

download_dir = "/content/drive/MyDrive/Colab Notebooks/ArtExtract/
↪downloaded_images"
os.makedirs(download_dir, exist_ok=True)

def download_image(url):

    image_id = url.split("/")[-1]

    image_url = f"{url}/full/full/0/default.jpg"
    filename = f"{image_id}.jpg"
    filepath = os.path.join(download_dir, filename)

    try:
        headers = {"User-Agent": "Mozilla/5.0"}
        r = requests.get(image_url, headers=headers, stream=True, timeout=20)
        r.raise_for_status()

        with open(filepath, "wb") as f:
            for chunk in r.iter_content(8192):
                if chunk:
                    f.write(chunk)

        print("Downloaded:", filename)

    except Exception as e:
        print("Failed:", image_id, e)


with ThreadPoolExecutor(max_workers=15) as executor:
    executor.map(download_image, image_urls)
```

Now, i've downloaded the images from the url locally, and added the local path in the dataset

[10]:
```
subset_df["image_path"] = subset_df["uuid"].apply(lambda x: f"downloaded_images/
↪{x}.jpg")
## keeping only the required useful columns
subset_df = subset_df[['uuid', 'title', 'attribution', 'classification',␣
↪'image_path']]
subset_df.to_csv('/content/drive/MyDrive/Colab Notebooks/ArtExtract/task2_df.
↪csv', index=False)
```

I'll save the final dataset locally so that it can be used again

[2]:
```
## final dataset
dataset = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/ArtExtract/
↪task2_df.csv')
```

[64]:
```
dataset[:5]
```

[64]:
```
                                   uuid                   title  \
0  20eb9317-4237-419b-91e8-3c5a568fc812           William Pitt
1  d068c7b1-0a96-4b46-95ef-bda8d4717552     Mattias de' Medici
2  818d138e-3c36-4d40-a169-e1528b712747       Trailing Arbutus
3  b347da91-8047-457f-8a44-949318dee128                Untitled
4  539d779f-65fc-4bec-abde-8ddd4d4ceab7  Corn and Winter Wheat

                   attribution classification  \
0          Gainsborough Dupont       Painting
1  Studio of Justus Sustermans       Painting
2              Fidelia Bridges       Painting
3               Robert Mangold       Painting
4           Thomas Hart Benton       Painting

                                          image_path
0  downloaded_images/20eb9317-4237-419b-91e8-3c5a…
1  downloaded_images/d068c7b1-0a96-4b46-95ef-bda8…
2  downloaded_images/818d138e-3c36-4d40-a169-e152…
3  downloaded_images/b347da91-8047-457f-8a44-9493…
4  downloaded_images/539d779f-65fc-4bec-abde-8ddd…
```

2 Now that my final dataset is prepared, i'll now move on to using deep learning model for feature extraction

Reference : Building an Embedding based image retrieval with ResNet50

Before processing, images are preprocessed to match ResNet's input requirements

[3]:
```python
## preprocessing the images
import torch
from torchvision import transforms
transform = transforms.Compose([
    transforms.Resize(256), # resize the shortest side
    transforms.CenterCrop(224), ## cropping to avoid distortion instead of␣
 ↪direct resizing to 224x224
    transforms.ToTensor(),
    transforms.Normalize(
        mean = [0.485,0.456,0.406],
        std = [0.229,0.224,0.225]
    ),
    transforms.Lambda(lambda x:x.unsqueeze(0))
])

from PIL import Image

def preprocess_image(image_path : str) -> torch.Tensor:
  image = Image.open(image_path).convert('RGB')
  img_tensor = transform(image)
  return img_tensor
```

[4]:
```python
# loading the resnet model
from torchvision.models import resnet50
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = resnet50()
feature_extractor = torch.nn.Sequential(*(list(model.children())[:-1]))
feature_extractor = feature_extractor.to(device)
feature_extractor.eval()
```

[4]:
```
Sequential(
  (0): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3),
bias=False)
  (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (2): ReLU(inplace=True)
  (3): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1,
ceil_mode=False)
  (4): Sequential(
    (0): Bottleneck(
      (conv1): Conv2d(64, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
```

```
      (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
      (downsample): Sequential(
        (0): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      )
    )
    (1): Bottleneck(
      (conv1): Conv2d(256, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
    (2): Bottleneck(
      (conv1): Conv2d(256, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
  )
  (5): Sequential(
    (0): Bottleneck(
      (conv1): Conv2d(256, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
```

```
      (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
      (downsample): Sequential(
        (0): Conv2d(256, 512, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      )
    )
    (1): Bottleneck(
      (conv1): Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
    (2): Bottleneck(
      (conv1): Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
    (3): Bottleneck(
      (conv1): Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
```

```
    )
  )
  (6): Sequential(
    (0): Bottleneck(
      (conv1): Conv2d(512, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(2, 2), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
      (downsample): Sequential(
        (0): Conv2d(512, 1024, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      )
    )
    (1): Bottleneck(
      (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
    (2): Bottleneck(
      (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
```

```
    (3): Bottleneck(
      (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
    (4): Bottleneck(
      (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
    (5): Bottleneck(
      (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
  )
  (7): Sequential(
    (0): Bottleneck(
      (conv1): Conv2d(1024, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(2, 2), padding=(1,
1), bias=False)
```

```
      (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
      (downsample): Sequential(
        (0): Conv2d(1024, 2048, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      )
    )
    (1): Bottleneck(
      (conv1): Conv2d(2048, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
    (2): Bottleneck(
      (conv1): Conv2d(2048, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
  )
  (8): AdaptiveAvgPool2d(output_size=(1, 1))
)
```

3 Now extracting the image embeddings

I've used a pretrained ***ResNet50*** model to extract visual features embeddings from the paintings and the final classification layer is removed.

moving the pytorch tensors to cpu memory is required because numpy cannot work on gpu tensors (converting tensor to array) and then flatten to 1d array

[5]:
```
import numpy as np
def extract_features(img_path : str) -> np.ndarray:
  img = preprocess_image(img_path)
  img = img.to(device)
  with torch.no_grad():
    features = feature_extractor(img)
    features = features.cpu().numpy().flatten()
  return features
```

Here, i've extracted the features from every image in the dataset and stored them in a ***features dictionary***

[6]:
```
## saving all the embeddings in a features dictionary
img_paths_from_dataset = dataset['image_path'].tolist()
features_dict = {
    path : extract_features(f"/content/drive/MyDrive/Colab Notebooks/ArtExtract/
 ↪{path}")
    for path in img_paths_from_dataset
}
```

[22]:
```
path_to_artist = dict(zip(dataset["image_path"], dataset["attribution"]))
```

3.1 Similarity Metrics :

- Cosine similarity
- Euclidiean distance

[28]:
```
## a function for returning cosine similarity and euclidean distances
from sklearn.metrics.pairwise import cosine_similarity, euclidean_distances
import numpy as np

def find_similar_images(query_img_path, num_results: int = 5, metric="cosine"):
    query_features = extract_features(query_img_path)
    similarities = []

    for path, features in features_dict.items():
        if path == query_img_path: ## excluding the query image so as to keep␣
 ↪the metrics unbiased
          continue

        if metric == "cosine":
            score = cosine_similarity(
                query_features.reshape(1, -1),
                features.reshape(1, -1)
            )[0][0]
```

```
        elif metric == "euclidean":
            score = euclidean_distances(
                query_features.reshape(1, -1),
                features.reshape(1, -1)
            )[0][0]

        artist = path_to_artist[path]

        similarities.append((path, artist, score))

    if metric == "cosine":
        similarities.sort(key=lambda x: x[2], reverse=True)
    elif metric == "euclidean":
        similarities.sort(key=lambda x: x[2])

    return similarities[:num_results]
```

3.1.1 Plotting the obtained similar images

[36]:
```
import matplotlib.pyplot as plt

query_img_path = '/content/drive/MyDrive/Colab Notebooks/ArtExtract/
 ↪downloaded_images/9a17a98d-5e76-4631-b4fa-003b99cc1af3.jpg'

cosine_results = find_similar_images(query_img_path, metric='cosine')
euclidean_results = find_similar_images(query_img_path, metric='euclidean')

plt.figure(figsize=(15, 8))

# a row for getting cosine similar images
for i, (path, artist, score) in enumerate(cosine_results):
    img = Image.open(f"/content/drive/MyDrive/Colab Notebooks/ArtExtract/
 ↪{path}")

    plt.subplot(2, len(cosine_results), i + 1)
    plt.imshow(img)
    plt.title(f"Cosine\n{artist}\n{score:.4f}")
    plt.axis("off")

# a row for getting euclidean distanced images
for i, (path, artist, score) in enumerate(euclidean_results):
    img = Image.open(f"/content/drive/MyDrive/Colab Notebooks/ArtExtract/
 ↪{path}")

    plt.subplot(2, len(euclidean_results), len(euclidean_results) + i + 1)
```

```
    plt.imshow(img)
    plt.title(f"Euclid\n{artist}\n{score:.4f}")
    plt.axis("off")

plt.tight_layout()
plt.show()
```

Here, the 1st image is the query image which has the highest similarity score of 1 and the lowest Euclidean distance of 0.

The other 4 are the images which have highest similarity scores less than 1 and lowest euclidean distances but greater than 0 (The query image itself)

3.1.2 Observations from the above example

The images retrieved by both cosine similarity and euclidean distance, have these features in common : - All are Portrait paintings - With similar head poses and faces - similar background or historical style or details

So from here, we can infer that the resnet embeddings are working well

- The cosine embeddings are very close to each other and therefore the portraits have very strong similar visual features
- The euclidean distances are far away after the 1st one which is the query image itself (distance is = 0) and then the values are 0, 31, 34 ……. and so on (still the retrieved images resemble similar visual features)

Below is another example of the similarity metrics output by the model

[40]:
```
import matplotlib.pyplot as plt

query_img_path = '/content/drive/MyDrive/Colab Notebooks/ArtExtract/
 ↪downloaded_images/bd5664bc-0ccf-4c93-99c7-188b75b2092d.jpg'

cosine_results = find_similar_images(query_img_path, metric='cosine')
euclidean_results = find_similar_images(query_img_path, metric='euclidean')

plt.figure(figsize=(15, 8))

# a row for getting cosine similar images
for i, (path, artist, score) in enumerate(cosine_results):
    img = Image.open(f"/content/drive/MyDrive/Colab Notebooks/ArtExtract/
 ↪{path}")

    plt.subplot(2, len(cosine_results), i + 1)
    plt.imshow(img)
    plt.title(f"Cosine\n{artist}\n{score:.4f}")
    plt.axis("off")

# a row for getting euclidean distanced images
for i, (path, artist, score) in enumerate(euclidean_results):
    img = Image.open(f"/content/drive/MyDrive/Colab Notebooks/ArtExtract/
 ↪{path}")

    plt.subplot(2, len(euclidean_results), len(euclidean_results) + i + 1)
    plt.imshow(img)
    plt.title(f"Euclid\n{artist}\n{score:.4f}")
    plt.axis("off")

plt.tight_layout()
plt.show()
```

3.1.3 Observations from the above example

For the given image which is a landscape painting with sky, trees, road and scenery,

- Cosine similarity : retrieved the images that include landscapes, outdoor scenes, similar sky or backgrounds
- Euclidean distance : gave more mixed results with one abstract painting and one portrait, an image with horses and animals (it's surprising here, as the query image also have horses in the scene), and an architectural scene

From here, it can be said that cosine similarity outperforms euclidean distance in similarity metrics because euclidean distance is less consistent for this task

One reason for the better performance of cosine metrics is that it inculcates the direction of the embedding vector whereas euclidean distance only focuses on the magnitude.

For ex : if we are give with two vectors [2, 2], and [4, 4]

Euclidean distance will only find the magnitude of distance between them

Whereas the cosine metric will understand that both of these tow vectors are projected in the same direction as well

So, in conclusion, images with similar visual features produces vectors pointing in similar directions, making cosine similarity more robust to these variations.

3.2 Evaluation Metrics

Moving on to evaluation metrics, i must define what 'relevance' is for the relevant images that are being retrieved

In this case, since the ground truth similar images are not available, i'll define two images similar if they have same artist *(because artists have the similar brushstrokes and artistic styles in almost all of their artworks)*

3.2.1 Metrics to be used :

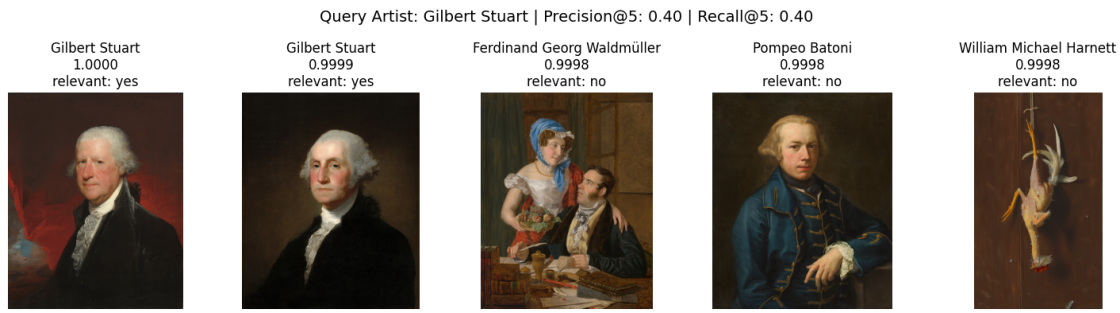
- Precision@5
- Recall@5
- mAP (mean Average Precision)

[25]:
```
import os

def visualize_retrieval(query_img_path, results, path_to_artist, k=5):

    query_key = os.path.relpath(query_img_path, "/content/drive/MyDrive/Colab␣
↪Notebooks/ArtExtract/")
    query_artist = path_to_artist[query_key]

    total_relevant = list(path_to_artist.values()).count(query_artist)

    top_k = results[:k]
    relevant_count = 0

    plt.figure(figsize=(15, 4))

    for i, (path, artist, score) in enumerate(top_k):
        img = Image.open(f"/content/drive/MyDrive/Colab Notebooks/ArtExtract/
↪{path}")

        ## checking if relevant or not
        if artist == query_artist:
            relevant_count += 1
            relevant = "yes"
        else:
            relevant = "no"

        plt.subplot(1, k, i + 1)
        plt.imshow(img)
        plt.title(f"{artist}\n{score:.4f}\nrelevant: {relevant}")
        plt.axis("off")

    ## calculating precision and recall
    precision = relevant_count / k
    recall = relevant_count / total_relevant

    plt.suptitle(
        f"Query Artist: {query_artist} | Precision@{k}: {precision:.2f} |␣
↪Recall@{k}: {recall:.2f}",
```

```
        fontsize=14
    )

    plt.tight_layout()
    plt.show()
```

3.2.2 Precision and recall for cosine

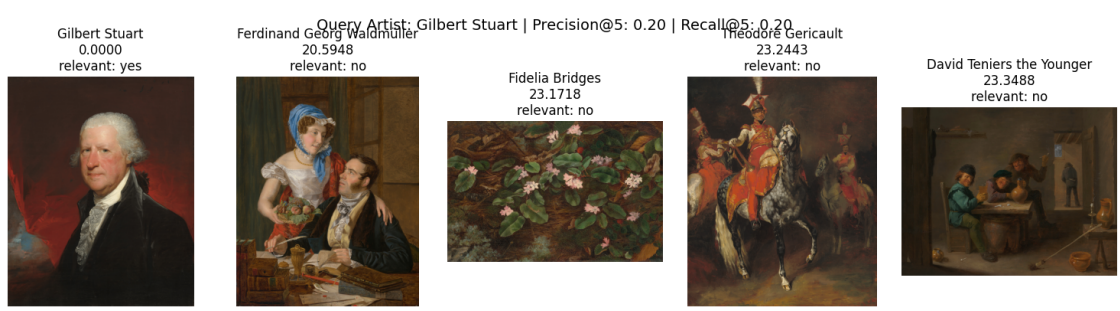
[58]:
```
query_img_path = '/content/drive/MyDrive/Colab Notebooks/ArtExtract/
 ↪downloaded_images/a88ccdd7-8743-4b6a-8419-91c64c7ddf8d.jpg'
cosine_results = find_similar_images(query_img_path, metric="cosine")
euclidean_results = find_similar_images(query_img_path, metric="euclidean")

visualize_retrieval(query_img_path, cosine_results, path_to_artist, k=5)
```



Here, only one correct image from the same artist is retrieved (still indicating that the model's extracted embeddings are well working)

3.2.3 Precision and recall for euclidean

[59]:
```
visualize_retrieval(query_img_path, euclidean_results, path_to_artist, k=5)
```



From the above, it can be realized that euclidean distance is ress reliable for semantic similarity

3.2.4 Overall conclusion

This task is performed successfully by using a pretrained ResNet50 model to extract visual embeddings and the results show that cosine similarity is more trustable and reliable in projects where similar compositions and fine details matter.

While, this model successfully caputres global features like similar poses, styles, color pallette, it ocassionlly retrieves visually related but images which are semantically different in their composition.

This task can also be improved with fine tuning of the model for such images or using better architectures like siamese networks for larger such datasets